Lecture 11 AOP (Aspect Oriented Programming)

- In simple term, It helps to intercept the method invocation. And we can perform some task before and after the method.

- AOP allow us on focus on bussiness logic by handling boilerplate and repetitive code like logging, transaction management etc.

- So, Aspect is a module which handle this repetitive or boilerplate code.

- Helps in achieving reusability, maintainbility of code.

Used during:

- Logging
- Transaction management
- Security etc.

Dependency you need to add in pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Concept && Coding

Demo

```
@RestController
@RequestMapping(value = "/api")
Public class Employee {
    @GetMapping(path = "/fetchEmployee")
    Public String fetchEmployee() {
        return " item fetched";
    }
}
```

```
@Component
@Aspect
Public class LoggingAspect {
  @Before("execution(public String con.recu..LoggingAspect)
  Public void beforeMethod() {
        S.O.P("inside beforeMethod Aspect"); logging
  }
}
```

Output :-

inside beforeMethod Aspect

item fetched.

* Before intercept the method and then fetch the employee.

Concept && Coding

How interception interanally work ??

```
@Component
@Aspect
Public
Public void beforeMethod ( ) {
    S.o.p("inside beforeMethod Aspect");
}
}
```

Some important AOP Concepts

Access-modifiers: optional and can be omitted

Return type: optional but can not be omitted

This is pointcut

```
@Componet
@Aspect
Public class LoggingAspect {
    @Before("execution(public String com.learning.Employee.fetchEmployee())")
    Public void beforemethod ( ) {
        System.out.println(" inside beforeMethod Aspect");
    }
}
```

This is Advice

Pointcut: Its an Expression, which tells where ADVICE should be applied.

Concept && Coding